

ENGINEERING BLUEPRINT

Enterprise AI Agent Platform

Product & Technical Blueprint

Source document: PRODUCT_VISION_AGENTS.pdf

Prepared from the perspectives of: Product Management, Business Analysis, Solution Architecture, QA

Scope: Engineering blueprint, sections A-O, analysis only

Date: 26 August 2026

Contents

Notation

0. Scope Note: Codebase Inspection Could Not Be Performed

A. Executive Summary

- A.1 What is being built
- A.2 The strategic reading
- A.3 The five decisions that determine whether the prototype survives to production
- A.4 Overall readiness

B. Product Modules

C. User Roles, Permissions & Lifecycle

- C.1 Roles
- C.2 The permission model
- C.3 The agent lifecycle as an explicit state machine

D. Agent Architecture

- D.1 The Agent Spec (the keystone artifact)
- D.2 AI Agent Designer (M3)
- D.3 Visual Agent Builder (M4)
- D.4 Agent Runtime (M5)
- D.5 Memory and context
- D.6 Guardrails and policies

E. MCP Architecture

- E.1 The three-level model
- E.2 The server manifest (how a new server is added with zero core changes)
- E.3 Transport adapters
- E.4 Credentials and isolation
- E.5 Dynamic discovery and schema drift
- E.6 Security specific to MCP
- E.7 How each stated requirement is met

F. Knowledge / RAG Architecture

- F.1 Pipeline
- F.2 Design decisions that matter

G. Enterprise / Governance Architecture

- G.1 Multi-tenancy
- G.2 Approval workflow
- G.3 Environments, deployment and rollback
- G.4 Audit logging
- G.5 Compliance posture

H. Technical Architecture

- H.1 Shape

H.2 Stack recommendations, with reasons

H.3 Deliberately not recommended yet

I. Data & API Model

I.1 Entities

I.2 Relationships, with the reasoning

I.3 API modules

J. Codebase Gap Analysis

J.1 Status: BLOCKED

J.2 What to provide

J.3 Classification rubric I will apply

J.4 The specific architectural risks I will test for

J.5 What I will deliver once unblocked

K. MVP / Phase 2 / Phase 3

K.1 A contradiction in the vision that must be resolved

K.2 MVP

K.3 Phase 2: make it sellable

K.4 Phase 3: make it a platform

L. Implementation Roadmap (dependency order)

L.0 Dependency graph

L.1 Workstreams

L.2 Continuous tracks (not sequential)

L.3 Sequencing rules I would enforce

M. Risks

M.1 Security risks

M.2 Technical risks

M.3 Product and UX risks

M.4 Scalability risks

M.5 Business risks

N. Open Questions

N.1 P0: block architecture or MVP scope

N.2 P1: block Phase 2 planning

N.3 Contradictions to resolve

N.4 Missing requirements (in the vision, not in this blueprint)

O. Recommended Next Steps

O.1 Immediate (this week)

O.2 Weeks 1 to 3 (W0)

O.3 Then follow L in dependency order

O.4 What can start today regardless of the open questions

O.5 What must NOT start until the questions are answered

Is the project sufficiently understood to start implementation?

Notation used throughout.

[STATED] explicitly written in the vision document · **[IMPLIED]** logically unavoidable given what is stated ·

[ASSUMPTION] needs business confirmation · **[RECOMMENDED]** my recommendation, not from the document ·

[GAP] missing, ambiguous or contradictory, needs a decision

Companion document: [Enterprise_AI_Agent_Platform_Plan.md](#) (the prior 20-area product analysis). This blueprint is engineering-facing and does not repeat that document; it cross-references it as **[REF: §n]** where the detail already exists, and goes deeper on MCP architecture, data model, API surface and dependency-ordered sequencing.

Notation

Tag	Meaning
[STATED]	Explicitly in the vision document
[IMPLIED]	Logically unavoidable given what is stated
[ASSUMPTION]	My assumption, requires confirmation before it becomes a requirement
[RECOMMENDED]	My recommendation, not from the document
[GAP]	Missing, ambiguous or contradictory in the source
[BLOCKED]	Cannot be completed with the inputs provided

0. Scope Note: Codebase Inspection Could Not Be Performed

[BLOCKED] The task asks me to inspect the existing codebase and produce a gap analysis. I checked the full environment. The only file provided is [PRODUCT_VISION_AGENTS.pdf](#). There is no repository, source tree, archive, database schema, [package.json](#), [pyproject.toml](#), Dockerfile or migration directory anywhere in the workspace.

I will not invent a gap analysis. Section **J** therefore contains, instead of findings: 1. the exact inspection protocol I will run once I have access, 2. the classification rubric I will apply, 3. the specific architectural risks I will be testing for, derived from this blueprint, 4. what to send me and how.

Everything else in this document is complete and independent of the codebase.

A. Executive Summary

A.1 What is being built

An enterprise, multi-tenant SaaS platform where a business user describes a process in natural language and receives a governed, observable, production-ready AI agent. The vision is explicit that this is **not** a chatbot, a workflow builder, a prompt manager, an API wrapper or a code generator (S22), but an **agent operating platform** combining creation, intelligence, connectivity, knowledge and operations.

A.2 The strategic reading

The builder is the demo. The operating layer is the product.

Natural-language agent creation is what wins the first meeting. It is also the most replicable part of the system; a competent team can clone a canvas and a generation prompt in a quarter. What cannot be cloned quickly is the layer beneath it: durable execution that survives a three-day approval wait, a complete and queryable trace of every decision, deterministic policy enforcement between the model and every tool call, and hard tenant isolation. The vision itself signals this in its two most specific sections, S15 (human-in-the-loop) and S16 (trust), which read like they came from real enterprise conversations rather than from product imagination.

Consequence for engineering: the foundation must be built operations-first. If the canvas ships before durable execution, tracing and isolation exist, the platform will require the rewrite S21 explicitly forbids.

A.3 The five decisions that determine whether the prototype survives to production

These are the irreversible ones. Everything else can be refactored.

#	Decision	Why it is irreversible
D1	The Agent Spec is the single source of truth. One versioned, schema-validated JSON document that the designer AI emits, the canvas renders, the runtime executes, the diff engine compares, the evaluator tests and the marketplace packages.	If the canvas and the runtime ever hold separate representations they will diverge, and "what you see is what runs" breaks. Reconciling two representations later is a rewrite of both.
D2	Durable, checkpointed execution from day one.	S15 requires approval timeouts and escalation, so executions live for days. Retrofitting durability onto a synchronous engine means rewriting every node executor.
D3	A Policy Decision Point between the model and every tool call. The model proposes; a deterministic engine authorizes.	Bolting authorization on later means auditing and modifying every call site, and until then the model is the security boundary, which is indefensible.
D4	Tenant isolation at the data layer (Postgres RLS, per-tenant vector namespaces, per-tenant encryption keys), not in application code.	Application-layer filtering depends on every developer remembering. S20 makes this a hard requirement, and a single breach is company-ending.
D5	Traces are structured product data with their own schema, retention and access control, not application logs.	S16 lists ten mandatory questions per execution. That is a queryable model, and reconstructing history from logs later is impossible.

A.4 Overall readiness

The vision is unusually clear about intent and unusually vague about several things that determine architecture. **Fifteen PO questions** must be answered before parts of the build can start (see **N**). Critically, though, they do

not all block equally: roughly 40% of the work (foundation, spec schema, execution engine, tenancy, MCP layer) can start immediately because it is invariant to those answers. See **●** for the precise split.

B. Product Modules

The vision's five layers (S4: Create / Operate / Discover over Models, Tools, RAG, Memory, Governance) decompose into ten engineering modules. This is the module boundary I recommend for the codebase, and it should also be the team boundary.

#	Module	Owns	Key surfaces
M1	Identity & Tenancy	Organizations, workspaces, users, roles, permissions, SSO, API keys, session and tenant context propagation	Admin UI, auth APIs
M2	Agent Definition	Agent Spec schema, drafts, versions, patches, validation, templates, diffing	Canvas, spec APIs
M3	Agent Designer (AI)	NL to spec generation, conversational patching, clarification, explanation, schema mapping, test generation	Chat panel in builder
M4	Visual Builder (Frontend)	Canvas rendering, node config, live execution overlay, diff review, TODO resolution	Web app
M5	Agent Runtime	Durable execution engine, node executors, state machine, retries, compensation, budgets	Workers, no direct UI
M6	Tools & MCP	Server catalog, connections, credential broker, transport adapters, discovery, tool grants, invocation	Admin UI, tool picker
M7	Knowledge & RAG	Knowledge bases, ingestion pipeline, chunking, embedding, retrieval, citations, permission mirroring	Knowledge UI
M8	Governance & Policy	Policy decision point, RBAC evaluation, approval workflow, audit log, guardrails	Admin UI, approval UI
M9	Observability & Cost	Trace capture and store, metrics, dashboards, alerting, token and cost accounting, usage metering	Monitoring UI
M10	Evaluation & Improvement	Test datasets, eval suites, scoring, deploy gating, feedback capture, improvement suggestions	Test UI

[RECOMMENDED] Module dependency rules, enforced by lint or build config: - M5 (Runtime) depends on M2, M6, M7, M8 but **never** on M3 or M4. The runtime must be able to execute a spec with no AI designer and no frontend present. This is what makes the platform testable and what makes the API-only path possible later. - M6 (Tools & MCP) exposes exactly one interface to M5. First-party connectors and third-party MCP servers are indistinguishable to the runtime. - M8 (Policy) is called by M5 and M6 but depends on neither, so it can be tested in isolation and audited independently. - M9 (Observability) is written to by everything and read by nothing in the hot path.

[STATED but deferred] Marketplace (S18, S19) is a Phase 3 module (M11) and is deliberately absent from the MVP module set. Its portability problem is unsolved [REF: §2.9].

C. User Roles, Permissions & Lifecycle

C.1 Roles

[GAP] The vision requires RBAC (S26) and permissions at six levels (S17) but names no roles. All roles below are **[RECOMMENDED]**.

Role	Scope	Can	Cannot	Rationale
Platform Super Admin	Vendor	Tenant lifecycle, fleet health, feature flags, consent-gated audited impersonation	Read tenant business data without break-glass	Required to operate the SaaS; [RECOMMENDED] , entirely absent from the vision
Org Owner	Tenant	Billing, SSO, workspaces, org policy, all roles, audit log	Bypass approval gates they configured	Buyer-side administrator
Workspace Admin	Workspace	Members, connections, knowledge, production deploy approval	Org billing or org policy	Departmental control
Agent Builder	Workspace	Create, edit, test agents; bind existing tools and knowledge; deploy to staging	Create credentials; deploy to production if gated; view secrets	The S30 primary user
Approver	Assignment	Decide runtime human tasks; optionally gate production deploys	Edit agents	S15 requires an approver role explicitly
Operator / SRE	Workspace	Monitor, pause, retry, bulk replay, manage alerts	Change agent logic	Separation of run from build
Auditor	Tenant	Read all traces, audit logs, grants; export evidence	Change anything	Compliance evidence gathering
Technical Steward	Workspace	Register MCP servers, define HTTP connectors, configure OAuth apps, manage secrets	(Usually also a Builder)	Resolves the S3-vs-S26 contradiction: this persona sees MCP, the Builder never does
End Consumer	Assignment	Trigger and converse with deployed agents; view own executions	See internals, others' executions, credentials	[GAP] their surface is undefined

C.2 The permission model

[STATED] S17 requires permissions at six levels: User → Workspace → Agent → Knowledge → Tool → Action.

[RECOMMENDED] Implement as (subject, action, resource, conditions) tuples evaluated by a central policy service, with three enforcement points:

1. API boundary	→ can this user call this endpoint on this resource?
2. Tool invocation	→ may THIS AGENT call THIS TOOL with THESE ARGUMENTS, in this environment, right now, within budget?
3. Database (RLS)	→ backstop; even a bug at layers 1 and 2 cannot cross tenants

Three rules that must be designed in from the start, not added later:

- **Effective-permission ceiling.** An agent acting for a user must never exceed that user's own permissions in downstream systems. Without this the platform is a privilege-escalation vector and will fail security review.

- **Runtime identity is explicit. [GAP, P0]** Whose credential does an agent use: a workspace service account, the builder's delegated token, or the invoking user's token? Default recommendation: workspace service connections, with optional per-user delegated auth as a per-connection setting.
- **Segregation of duties.** The requester of an action cannot approve it; validated at deploy time, not discovered at runtime.

C.3 The agent lifecycle as an explicit state machine

[STATED] S10 defines: Create → Design → Configure → Test → Evaluate → Approve → Deploy → Run → Monitor → Improve → Version → Redeploy.

[RECOMMENDED] Implement this as two separate state machines, because conflating them is a common and costly modelling error:

Definition lifecycle (an Agent and its Versions):



Rules: drafts are mutable and per-agent-singleton; versions are immutable and never edited; a deployment references exactly one version in exactly one environment; rollback creates a new deployment record pointing at an older version rather than mutating history.

Execution lifecycle (a single run):



Rules: every transition is checkpointed durably; every suspension records why, what it is waiting for, and its deadline; an execution pins its `agent_version_id` immutably at start, so a trace remains interpretable months later even though the agent has changed.

[RECOMMENDED] Where the vision's lifecycle is under-specified: "Evaluate" and "Improve" are named but never defined [REF: §2.6, §18 Q10]. Until product answers those, model them as states with pluggable implementations rather than hard-coding a meaning.

D. Agent Architecture

D.1 The Agent Spec (the keystone artifact)

Everything in the platform reads or writes this one document. **[RECOMMENDED]** Design and freeze the v1 schema before any other code is written.

```

{
  "specVersion": "1.0",          // schema version, enables migration
  "agent": {
    "name": "Employee Onboarding Agent",
    "description": "...",
    "criticality": "high",        // drives default approval policy
    "outputSchema": { /* JSON Schema */ }
  },
  "model": {
    "provider": "anthropic",      // abstracted, never hard-coded in nodes
    "model": "claude-sonnet-4-6",
    "modelPin": "2026-05-01",    // pinned; upgrades are releases
    "temperature": 0.2,
    "fallback": { "provider": "openai", "model": "..." }
  },
  "budgets": {
    "maxSteps": 40, "maxToolCalls": 25,
    "maxCostUsd": 2.00, "maxWallClockSec": 300,
    "approvalTimeoutSec": 259200  // 3 days
  },
  "triggers": [
    { "id": "t1", "type": "manual" },
    { "id": "t2", "type": "schedule", "cron": "0 7 * * 1-5", "timezone": "Asia/Kolkata" }
  ],
  "nodes": [
    { "id": "n1", "type": "trigger", "config": { "triggerId": "t2" } },
    { "id": "n2", "type": "tool", "config": {
      "toolRef": "tool_hris_get_employee", // reference, never a credential
      "argumentMapping": { "employeeId": "$.trigger.employeeId" },
      "dryRunBehavior": "mock"
    } },
    { "id": "n3", "type": "agent", "config": {
      "instructions": "Validate the submitted documents against policy.",
      "allowedTools": ["tool_doc_extract"],
      "outputSchema": { /* ... */ }
    } },
    { "id": "n4", "type": "knowledge", "config": {
      "knowledgeBaseIds": ["kb_hr_policies"], "topK": 6, "requireCitations": true
    } },
    { "id": "n5", "type": "condition", "config": {
      "expression": "$.n3.riskScore > 0.7" // deterministic, not model-decided
    } },
    { "id": "n6", "type": "approval", "config": {
      "approverRole": "hr_manager",
      "timeoutSec": 259200,
      "escalation": [{ "afterSec": 86400, "toRole": "hr_director" }],
      "onTimeout": "reject", // MUST be an explicit choice
      "requireReasonOnReject": true,
      "consequenceTemplate": "This will create an employee record in Workday."
    } },
    { "id": "n7", "type": "notification", "config": { /* ... */ } }
  ],
  "edges": [
    { "from": "n1", "to": "n2" },
    { "from": "n5", "to": "n6", "when": "true" },

```

```

    { "from": "n5", "to": "n7", "when": "false" }
  ],
  "bindings": {
    "tools": ["tool_hris_get_employee", "tool_hris_create_employee"],
    "knowledge": ["kb_hr_policies"]
    // connections resolved PER ENVIRONMENT at deploy time, never in the spec
  },
  "guardrails": {
    "piiRedaction": true,
    "blockedTopics": [],
    "injectionDetection": true
  }
}

```

Seven non-obvious rules that make this schema work:

1. **No credentials, ever.** Only references. This makes specs safely diffable, exportable and marketplace-packageable.
2. **No connection IDs in the spec.** The spec binds to *tool references*; the *environment* maps tool references to connections. This is what makes staging-to-production promotion possible without editing the spec. Getting this wrong is the most common cause of "it worked in staging."
3. **Conditions are deterministic expressions, not model calls.** This gives the user a predictable skeleton with model-driven variability confined to `agent` nodes, which directly addresses the S23 tension between deterministic workflows and autonomous reasoning.
4. `allowedTools` is **per-node, not just per-agent**. A reasoning node should only see the tools it needs, which improves selection accuracy and shrinks the injection blast radius.
5. **Budgets are in the spec, not in global config.** They are part of what gets versioned, reviewed and approved.
6. `onTimeout` **has no default**. Forcing an explicit choice at design time prevents the worst class of production surprise.
7. `specVersion` **from day one**, with a migration function per version bump. Old versions must remain executable forever, because traces reference them.

D.2 AI Agent Designer (M3)

Generation pipeline:

```

User description
↓
[1] Intent classification + ambiguity scoring
    ↓ (if ambiguous) → up to 3 clarifying questions, then continue
[2] Pattern retrieval: 3-5 similar reference agents from a curated library
    ↓
[3] Context assembly: node type schemas + the tenant's ACTUAL tool catalog
                      + available knowledge bases + org policy constraints
    ↓
[4] Constrained generation against the Agent Spec JSON Schema (streaming)
    ↓
[5] Validate: schema ✓ | tool refs exist ✓ | DAG acyclic ✓ | all nodes reachable ✓
              | policy-required approvals present ✓
    ↓ (on failure) → repair loop with errors fed back, max N attempts
[6] Emit spec + per-node rationale + list of unresolved TODOs

```

Design rules: - **The model may never invent a tool.** It selects from the supplied catalog or emits an `unconfigured_tool` node carrying a capability description. A generation that fails because a tool is missing is a

product failure; a generation that produces a TODO badge is a product success. - **Never a dead end.** Always return something on the canvas. - **Quality comes from the pattern library, not from the prompt.** Curated reference agents do the heavy lifting; raw model capability is the fallback, not the strategy. - **Stream it.** 20 to 60 seconds of silence reads as broken; 20 to 60 seconds of nodes appearing reads as thinking.

Conversational editing must emit patches, never regenerate:

```
{ "patches": [
  { "op": "insertNodeBefore", "target": "n5",
    "node": { "id": "n4b", "type": "approval", "config": {...} } },
  { "op": "setEdge", "from": "n4", "to": "n4b" }
],
"summary": "Added a manager approval before the employee record is created.",
"destructive": false
}
```

Full regeneration silently destroys user configuration and is the single largest source of trust loss in products of this shape. Patches give a reviewable diff, reversibility, interleaving of AI and manual edits, and a clean audit trail of who changed what. Destructive patches must be flagged and confirmed explicitly.

D.3 Visual Agent Builder (M4)

[STATED] S13 requires the canvas to feel alive: node states must stream during execution. S11 requires "Describe → See → Edit → Test → Deploy", not 25 technical settings.

[RECOMMENDED] Progressive disclosure by persona. Default view shows business-level configuration only. An "Advanced" panel, visible to the Technical Steward role, exposes model parameters, raw prompts, retry policy and argument mappings. This is how you satisfy S11 and S26 simultaneously rather than choosing between them.

Live execution: server pushes node state transitions over SSE (`pending` | `running` | `completed` | `failed` | `waiting_human` | `skipped`), keyed by `executionId` and `nodeId`. Polling will not deliver the S13 experience. Reconnection must replay missed events from the trace, or a dropped connection leaves the canvas permanently stale.

The TODO pattern: every unresolved requirement (unconnected tool, unselected knowledge base, unassigned approver) renders as a badge on the node with a one-click resolution path, or a "Request from admin" flow if the user lacks permission. This turns the generation-to-runnable cliff into a checklist and is, in my view, the highest-leverage UX decision in the entire product.

D.4 Agent Runtime (M5)

Execution loop:

```
Trigger → create Execution (pin agent_version_id, resolve environment)
  → durable workflow starts
  → for each node in topological order:
    checkpoint(state)
    executor = registry[node.type]
    result = executor.run(node, context) ← idempotent, retryable
    emit TraceSpan
    checkpoint(state, result)
  → on approval node: register timer, emit ApprovalRequest, SUSPEND
  → on external signal: RESUME from checkpoint
  → on failure: retry policy → compensation → dead-letter → alert
  → finalize: cost rollup, status, notification
```

Requirements on every node executor, enforced by an interface contract:

- **Idempotent.** A worker can die after a side effect but before the checkpoint; re-execution must not duplicate the effect. Implement with an idempotency key derived from `(executionId, nodeId, attempt)` passed to downstream systems where they support it.
- **Declares its side-effect class:** `read | write | irreversible`. This single field drives dry-run behavior, default approval policy and risk scoring. Adding it later means reclassifying every tool.
- **Emits a trace span** with input, output, model, tokens, cost, latency, and the policy decision.
- **Respects budgets** and fails fast when exceeded.

[RECOMMENDED] Compensation is a first-class concept. Write actions against external systems are not transactional. If step 6 fails after step 4 created an employee record, there is no rollback. The runtime must support a declared compensating action per node and, where none exists, must halt and escalate to a human with the exact external state described. This is the highest-consequence runtime edge case and it must be in the engine's design, not patched in later.

D.5 Memory and context

[GAP, P0] S4 lists Memory as one of five core primitives and S26 puts it in the first release, but the document never defines it. At least four different things are called "memory" in this category, with different data models, retention rules and GDPR implications:

Kind	Scope	Lifetime	Notes
Execution scratchpad	One run	Run duration	Not really memory; already in execution state
Conversation history	One conversation	Session or configurable	Needed if agents are chat-surfaced
Entity memory	Cross-execution, keyed to a business entity ("this vendor pays late")	Long	High value, highest privacy risk
Semantic memory	Cross-execution, embedded	Long	Substantially overlaps RAG; risks duplicating the knowledge layer

[RECOMMENDED] Ship MVP with execution scratchpad and conversation history only (both trivially derivable from data you already need), and defer entity and semantic memory to Phase 2 pending a product decision. Model the store behind an interface now so adding kinds later is additive.

D.6 Guardrails and policies

[STATED as a first-release item in S26, never defined.]

[RECOMMENDED] Two distinct mechanisms, often confused:

- **Guardrails** are content controls on the data path: PII detection and redaction, prompt-injection detection, blocked topics, output schema validation. Implemented as fast, cheap classifiers running inline on inputs and outputs. Cheap model, not the main agent.
- **Policies** are authorization controls on the action path: which tools, which actions, which arguments, which approvals, which budgets. Implemented deterministically in the Policy Decision Point (M8). No model involvement whatsoever.

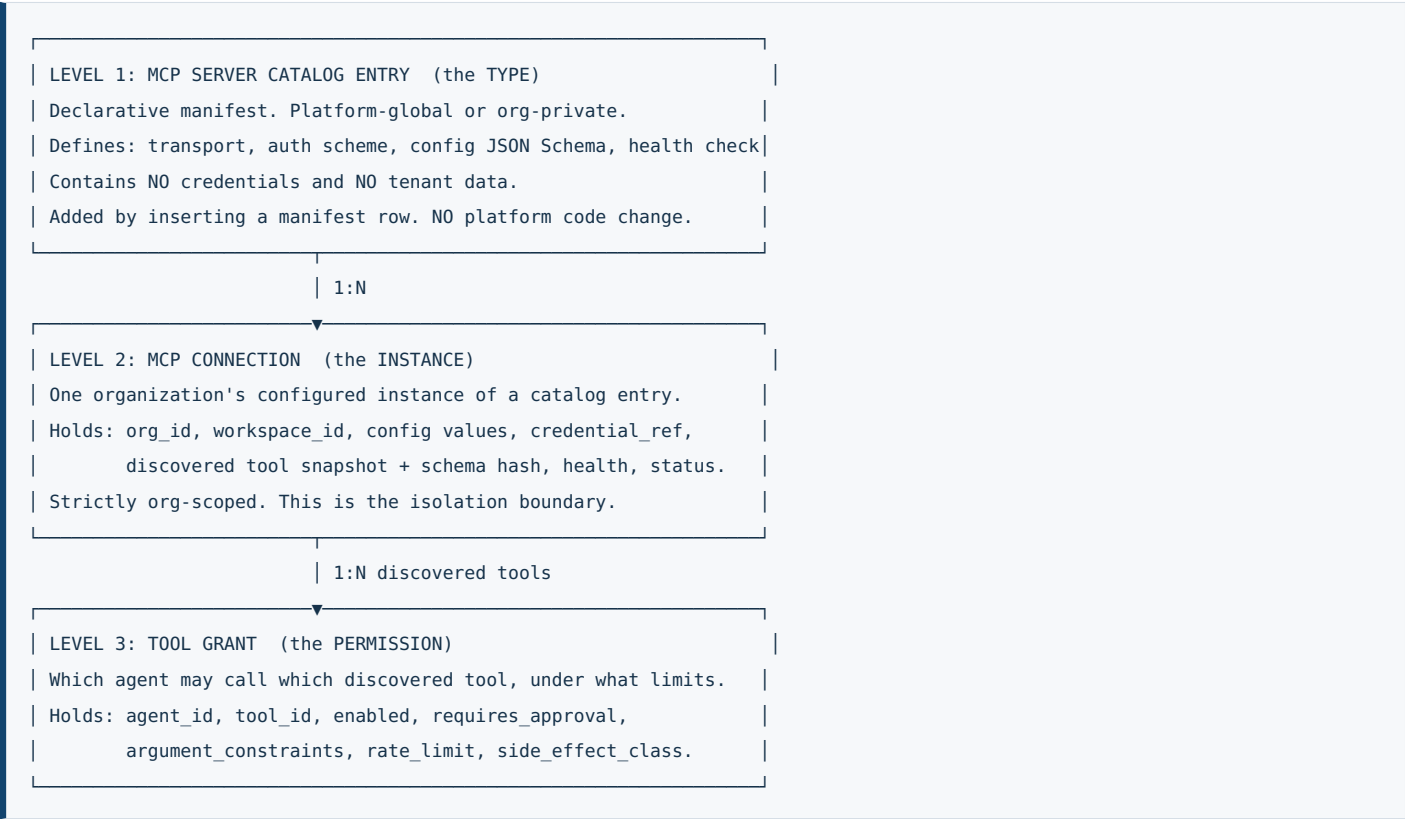
Keeping these separate matters because guardrails are best-effort and probabilistic, while policies must be exact and auditable. Conflating them produces a system where security depends on a classifier's recall, which is not a defensible position in an enterprise security review.

E. MCP Architecture

This is the section with the most specific requirements, so it gets the most specific design. The eight stated constraints are addressed individually in E.7.

E.1 The three-level model

The single most important structural decision: separate **type**, **instance** and **grant**. Most implementations conflate the first two and then cannot support per-client credentials, and conflate the second and third and then cannot enable tools per agent.



E.2 The server manifest (how a new server is added with zero core changes)

A catalog entry is data, not code:

```

{
  "key": "github",
  "displayName": "GitHub",
  "category": "developer",
  "visibility": "platform",          // or "org" for customer-private servers
  "transport": {
    "type": "streamable_http",      // stdio | streamable_http | sse | in_process
    "urlTemplate": "https://api.githubcopilot.com/mcp/",
    "timeoutMs": 30000,
    "maxResponseBytes": 262144
  },
  "auth": {
    "strategy": "oauth2_authorization_code",
    "authorizationUrl": "https://github.com/login/oauth/authorize",
    "tokenUrl": "https://github.com/login/oauth/access_token",
    "scopes": ["repo", "read:org"],
    "pkce": true,
    "placement": { "in": "header", "name": "Authorization", "format": "Bearer {token}" },
    "refresh": { "supported": true, "leadTimeSec": 300 }
  },
  "configSchema": {                // rendered as an admin form automatically
    "type": "object",
    "properties": { "organization": { "type": "string", "title": "GitHub org" } },
    "required": ["organization"]
  },
  "egressAllowlist": ["api.github.com", "api.githubcopilot.com"],
  "healthCheck": { "method": "tools/list", "intervalSec": 900 },
  "rateLimit": { "perMinute": 120, "burst": 20 }
}

```

This is the mechanism that satisfies "new MCP servers can be added without changing the core platform." Adding a server means writing a manifest. The admin form is generated from `configSchema`. The auth flow is selected from a fixed set of pluggable strategies. The transport is selected from a fixed set of adapters. No new code paths.

[RECOMMENDED] Allow exactly one escape hatch: an optional `adapter` field naming a registered custom auth or request-signing module, for servers with non-standard schemes (AWS SigV4, proprietary HMAC). This must be platform-code, admin-approved, and rare. Without it you will eventually be blocked; with it unguarded, the "no core change" property erodes.

E.3 Transport adapters

One interface, four implementations:

```

interface McpTransport {
  connect(connection): Session
  listTools(session): ToolDefinition[]
  listResources(session): Resource[]    // feeds the Knowledge layer
  listPrompts(session): PromptTemplate[]
  callTool(session, name, args, ctx): ToolResult
  close(session)
  health(session): HealthStatus
}

```

Transport	Use	Critical constraints
<code>streamable_http</code>	The primary remote transport, current MCP standard	Connection pooling, per-connection circuit breaker, response size cap, strict timeout
<code>sse</code>	Legacy remote servers	Support for compatibility only; do not build new capability on it
<code>stdio</code>	Servers distributed as a local process	Must run in a per-execution sandboxed container: no host filesystem, no network except the egress allowlist, CPU and memory caps, killed at execution end. In multi-tenant SaaS this is the highest-risk transport and should be admin-approved per server
<code>in_process</code>	First-party connectors implemented as native modules	[RECOMMENDED] This is the trick that lets first-party tools use the identical interface, policy path and trace format as third-party MCP servers without paying network cost. One tool abstraction across the whole platform

[RECOMMENDED] MCP `resources/list` should be surfaced to the Knowledge module (M7) as an ingestible source type, and `prompts/list` to the Designer (M3) as reference patterns. Most implementations only use `tools/*` and leave real capability unused.

E.4 Credentials and isolation

Credential storage:

KMS root key

└─ per-organization Data Encryption Key (DEK)

└─ envelope-encrypted credential blob (per connection)

Runtime access path:

Node executor requests tool call

→ Policy Decision Point authorizes

→ Connection Resolver resolves connection BY TENANT CONTEXT ONLY

→ Credential Broker decrypts just-in-time, INJECTS at the transport layer

→ the credential value is NEVER returned to the caller, the agent, the model context, the trace, or any log

→ post-call: zeroize in memory

Isolation invariants that must be enforced and tested continuously:

Invariant	Enforcement
A connection is reachable only from executions in its own org	Tenant context is derived from the execution record, never from a parameter; Postgres RLS as the backstop
Credentials are never in the LLM context	Structural: the model receives tool <i>schemas</i> , the transport receives credentials. They never meet
Credentials are never in traces	Redaction at span-write time, plus automated secret-scanning of stored spans in CI
One org's DEK compromise does not expose another org	Per-org DEKs, rotatable independently
Revoked consent propagates immediately	Revocation marks the connection dead; in-flight executions suspend with <code>SUSPENDED_CREDENTIALS</code> rather than failing

Credential ownership modes (per connection, chosen at creation): - `org_service` (default): one workspace-owned credential, all agents share it. Simple, auditable, but every action appears as the service account in the

downstream system. - `per_user_delegated` : each end user authorizes individually; the agent acts as the invoking user. Correct permission semantics and better downstream audit, but higher friction and more tokens to manage.

[GAP, P0] The vision does not address this choice. It has to be answered because it changes the connection data model, the OAuth flow and the audit story.

E.5 Dynamic discovery and schema drift

```
On connection create → tools/list → persist ToolDefinition snapshot
                        + content hash per tool + manifest version
On schedule (15 min) → tools/list → compare hashes
On drift detected    → connection.status = SCHEMA_CHANGED
                        → notify Technical Steward
                        → NEW tools appear as "pending review", NOT auto-enabled
                        → CHANGED tools: deployed agents continue on the PINNED
                           schema; the change requires explicit re-approval
                        → REMOVED tools: dependent agents flagged as at-risk,
                           surfaced in the agent registry before they fail
```

[RECOMMENDED] Deployed agents pin the tool schema version they were validated against. Auto-adopting upstream schema changes into production agents is convenient and wrong: it means a third party can silently alter what runs inside a customer's tenant, which is unacceptable in an enterprise context and undermines the S16 trust guarantee.

E.6 Security specific to MCP

This is a real and under-appreciated attack surface.

Threat	Mechanism	Mitigation
Tool-description injection	Tool names and descriptions enter the model's context. A malicious or compromised server can embed instructions there	Treat descriptions as untrusted data: sanitize, length-cap, strip instruction-like patterns, wrap in clearly-delimited data blocks. Require admin review of descriptions at connection approval, and re-review on drift
Rug-pull	A server behaves benignly during review, then changes tool behavior later	Schema pinning, drift alerts, response anomaly detection, and periodic re-review of high-privilege connections
Confused deputy	Agent A in org X reaches a connection belonging to org Y	Tenant context derived from the execution only; runtime assertion before every call; RLS backstop; automated cross-tenant probes in CI
Data exfiltration via tool arguments	An injected instruction causes the agent to pass sensitive context as an argument to an attacker-controlled tool	Egress allowlist per connection, argument constraints in the tool grant, PII detection on outbound arguments, approval requirement on tools classified irreversible
Resource exhaustion	A server returns a 500 MB response or hangs	Hard response size cap, strict timeout, circuit breaker, per-connection rate limits
Malicious stdio server	Arbitrary code execution in the platform	Sandboxed container, no host FS, no default network, resource caps, admin approval required before any stdio server is usable

E.7 How each stated requirement is met

Requirement	Mechanism
Admin can register MCP servers	Catalog entry (Level 1), created via admin UI or manifest import; <code>visibility: org</code> for customer-private servers
Different transports supported	Four pluggable <code>McpTransport</code> adapters selected by manifest, no caller changes
Clients can connect their own credentials	Connection (Level 2) is org-scoped; each org supplies its own config and credentials against the same catalog entry
OAuth / API keys / secrets supported	Pluggable auth strategies: <code>none</code> , <code>api_key</code> , <code>bearer_static</code> , <code>basic</code> , <code>oauth2_authorization_code</code> , <code>oauth2_client_credentials</code> , <code>mcp_oauth</code> (dynamic client registration), <code>mtls</code> , <code>custom_adapter</code>
Tools discovered dynamically	<code>tools/list</code> on create and on schedule, snapshotted with content hashes, drift-detected
Tools enabled/disabled per agent	Tool Grant (Level 3): explicit per-agent allow-list with per-tool constraints
Credentials isolated per org/client	Per-org DEK under a KMS root, connection rows org-scoped with RLS, credential values never leave the transport layer
New servers without core changes	Declarative manifest + generated config form + fixed adapter sets. Adding a server is a data operation

F. Knowledge / RAG Architecture

[STATED] S8: knowledge bases are reusable across agents; the pipeline is Documents → Processing → Chunking → Embeddings → Retrieval → Agent.

F.1 Pipeline

```
Source (upload | connector sync | MCP resource)
  ↓ checksum, dedupe, virus scan
Parse (PDF, DOCX, XLSX, HTML, MD; OCR fallback for scanned PDFs)
  ↓ structure-aware, preserving headings and tables
Chunk (structure-aware, with overlap; table rows kept intact)
  ↓
Enrich (title, section path, source URL, effective date, ACL metadata)
  ↓
Embed (batched, model + version recorded per chunk)
  ↓
Index (per-tenant namespace, hybrid: vector + lexical)
  ↓
Retrieve (query rewrite → hybrid search → rerank → top-k → citations)
```

F.2 Design decisions that matter

- **Per-tenant namespace, filtered at the store, not after retrieval.** Post-retrieval filtering is a leak waiting for a bug. This is a hard S20 requirement.
- **Record the embedding model and version on every chunk.** Changing embedding models forces a re-index; without per-chunk versioning that migration is a full outage rather than a rolling background job.
- **Effective dates on documents.** When an old and a new policy both match, retrieval must surface both with dates and the agent must flag the conflict rather than silently picking one. For HR and Legal use cases this is a correctness requirement, not a nicety.
- **Citations are mandatory when knowledge is used.** Required to make S16 trust real, and the cheapest defence against hallucination complaints.
- **"I don't know" is a first-class behavior.** When retrieval returns nothing above threshold, the agent must say so. This belongs in the node executor's contract, not in a prompt.
- **[RECOMMENDED] Permission mirroring is Phase 2 and genuinely hard.** If a user cannot read a SharePoint document, the agent must not surface its content to them. Options: sync source ACLs into chunk metadata and filter at query time (accurate, complex, needs continuous re-sync), or scope whole knowledge bases by role (crude, simple, adequate for MVP). Start with the second; be honest that it is a limitation.

[GAP] The vision never states whether knowledge is org-scoped or workspace-scoped. Recommendation: workspace-scoped with optional org-wide sharing, which matches "one knowledge base, many agents" (S8) without making all HR documents visible to Finance.

G. Enterprise / Governance Architecture

G.1 Multi-tenancy

[STATED] S20: no tenant may ever access another tenant's agents, knowledge, credentials, executions, users, logs or business data.

[RECOMMENDED] Shared infrastructure, isolated at the data layer (single database, RLS enforced), for these reasons: one schema to migrate, one deployment to operate, and per-tenant physical separation is a Phase 3 concern for regulated customers rather than a Day 1 default.

```
Request → authenticate → resolve tenant from the SESSION, never from a parameter
    → SET LOCAL app.current_org_id = '<uuid>' (transaction-scoped)
    → every tenant-scoped table has an RLS policy USING (org_id = current_setting(...))
    → vector store: namespace per org
    → object storage: prefix per org, IAM-scoped
    → credentials: DEK per org
    → queues: per-org concurrency caps and fair scheduling
```

[RECOMMENDED] A cross-tenant isolation test suite that runs in CI on every commit, attempting access to a second tenant's resources through every API, every retrieval path, every trace query and every tool invocation. Isolation cannot be verified by code review. This suite is the single highest-value piece of test infrastructure in the project.

[GAP, P0] Is shared-infrastructure multi-tenancy acceptable to the target customers, or will regulated buyers require dedicated infrastructure or VPC deployment? This changes the deployment architecture fundamentally and should be asked of design partners in week one.

G.2 Approval workflow

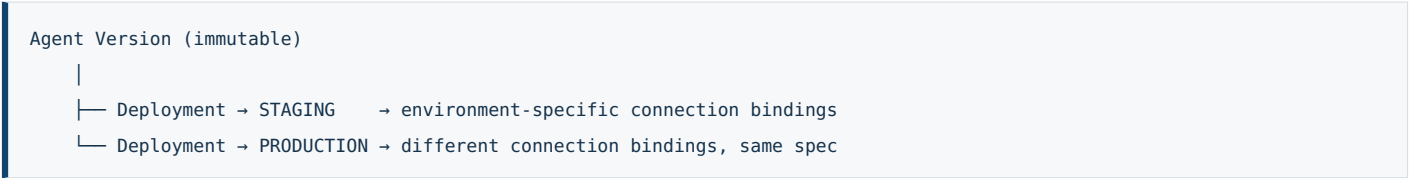
[STATED] S15 specifies, unusually precisely: Approver, Role, Timeout, Escalation, Approval conditions, Rejection path, Audit trail. All seven are MVP.

```
Runtime reaches approval node
    → evaluate condition (skip if not met)
    → resolve approvers (named user | role membership | dynamic expression)
    → validate: at least one approver exists AND requester is not the sole approver
    → create ApprovalRequest (context snapshot, proposed action, CONSEQUENCE text,
                             deadline, escalation chain)
    → SUSPEND execution durably; register a timer
    → notify via configured channels; deep link is SIGNED, SINGLE-USE, EXPIRING
    → on decision: verify token, verify not already decided, record immutably
                   (approver, timestamp, agent_version, payload hash, what they saw)
    → SIGNAL the execution → resume on the approved or rejected branch
    → on timeout: escalate → escalate → apply the spec's explicit onTimeout action
```

[RECOMMENDED] The approver must see the consequence, not the payload. "Approve this JSON" is compliance theatre and enterprises recognise it as such. The spec carries a `consequenceTemplate` for exactly this reason. This is where trust is won or lost and it deserves dedicated design work.

Edge cases that must be handled in v1, not discovered in production: approver has left the company (deprovisioning reassigns pending approvals), stale link clicked after a decision (show the existing decision, never double-execute), zero role members (blocked at pre-deploy validation), notification delivery failure (fallback channel plus alertable event, because an undelivered approval is a silently stuck execution).

G.3 Environments, deployment and rollback



- **Promotion changes the deployment record, never the spec.** The environment supplies the connection mapping. This is what makes "tested in staging" mean anything.
- **Rollback creates a new deployment pointing at an older version.** History is never mutated.
- **In-flight executions complete on their pinned version.** New executions use the current one.
- **Pre-deploy validation gate** (S14 shows this explicitly): schema valid, all tool refs resolvable in the target environment, all credentials live, no unreachable nodes, approvers exist, budgets set, required approvals present per policy. Each failure links to the offending node.
- **[RECOMMENDED]** Production Deploy can require human approval, configurable per workspace. This maps directly to the S10 "Approve → Deploy" step and to enterprise change-management expectations.

G.4 Audit logging

Append-only, tamper-evident (hash-chained is worth the small cost given the compliance positioning), covering: authentication, permission changes, connection and credential lifecycle, MCP server registration and approval, agent create/edit/version/deploy/rollback, every approval decision, every tool invocation with its policy decision, data exports, and support impersonation.

[RECOMMENDED] Keep the audit log separate from the execution trace. They have different consumers (compliance vs debugging), different retention, different access control and different volumes. Conflating them produces a trace UI that leaks internals and an audit log too noisy to audit.

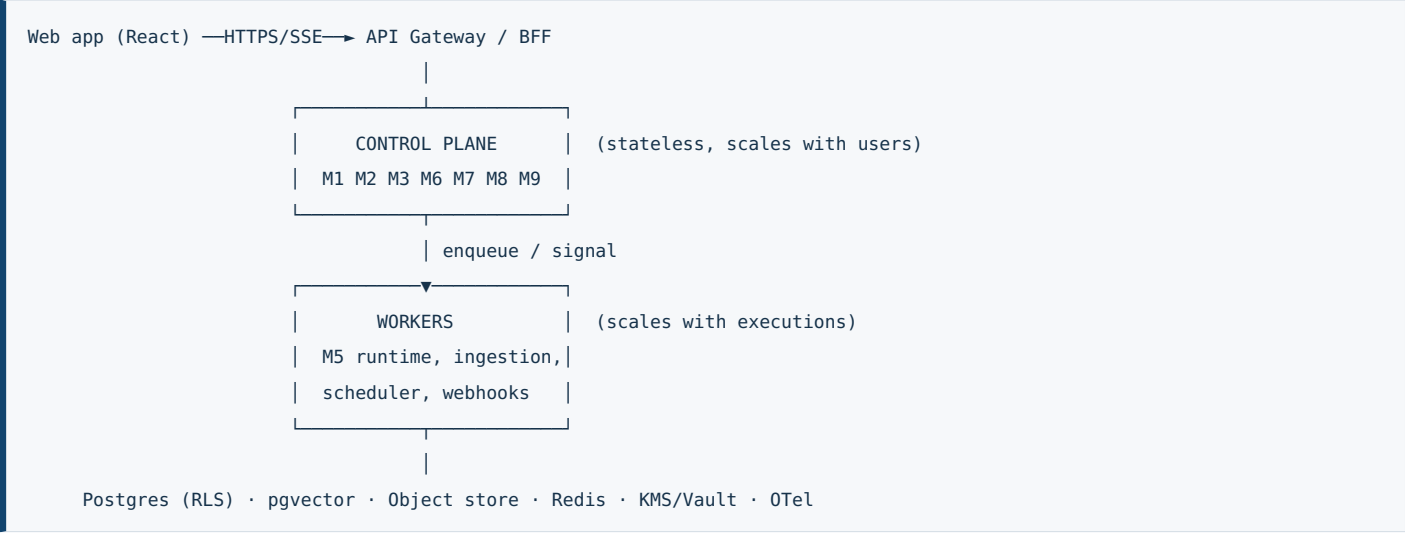
G.5 Compliance posture

[GAP] Not mentioned anywhere in the vision, yet SOC 2 Type II is a near-universal prerequisite for enterprise purchase, and finance, HR and legal use cases pull in GDPR and possibly SOX. Compliance shapes architecture (audit, retention, encryption, access control, sub-processor management), so scope it now even if certification comes later. Also worth legal review: automated decisions affecting individuals (hiring, loans) fall under EU AI Act scrutiny, which the human-in-the-loop design already largely anticipates.

H. Technical Architecture

H.1 Shape

[RECOMMENDED] A modular monolith split into two deployables: a synchronous control plane and asynchronous workers. Microservices at this stage add distributed-systems cost with no benefit, and the S21 "no rewrite" requirement is satisfied by clean module boundaries (B), not by network boundaries. The boundaries in B are exactly where you would later split.



H.2 Stack recommendations, with reasons

Layer	Recommendation	Reason (not preference)
Frontend	React + TypeScript; React Flow (or equivalent) for the canvas; SSE for live execution; TanStack Query for server state	The canvas is the product. Hand-rolling a graph editor is months of undifferentiated work. Live node state (S13) requires push
Backend	Modular monolith, TypeScript or Python end to end	[ASSUMPTION] team composition unknown. Choose by existing team strength, not fashion. A TS control plane with Python workers is defensible but doubles the toolchain and the type definitions; only do it if the AI work genuinely needs the Python ecosystem
Database	PostgreSQL: RLS for isolation, JSONB for specs, native partitioning for executions and trace spans	One store covering relational, document and (via pgvector) vector keeps the prototype simple and remains production-viable. Traces will be the largest table by an order of magnitude, so partition from day one
Vector	pgvector behind a repository interface	Avoids a second datastore early; the interface is cheap now and expensive later. Migrate on measured need, not anticipation
Durable execution	Temporal, or a Postgres-backed state machine with a correct checkpoint model	Decide in week one (D2). Temporal is operationally heavier but correct; a custom engine is lighter now and risks re-implementing Temporal badly. Either is defensible; deferring the decision is not
Queue / cache / pubsub	Redis	Needed anyway for SSE fan-out, rate limiting and idempotency keys

Layer	Recommendation	Reason (not preference)
AI layer	Provider-abstracted model router: Anthropic, OpenAI, Azure OpenAI, Bedrock. Per-tenant keys, model pinning, streaming, retries, fallback, token accounting	Enterprises will demand specific providers, regions and no-training guarantees. Hard-coding one provider is a commercial dead end
MCP layer	As designed in E . All tools, including first-party, invoked through it via the <code>in_process</code> transport	One invocation path, one policy point, one trace format
Auth	OIDC sessions; SAML/OIDC SSO + SCIM for enterprise (Phase 2); scoped API keys; signed single-use tokens for approval links	An approval link that can be replayed or forwarded is a governance failure
Secrets	Cloud KMS or Vault, envelope encryption, per-org DEKs	S20 lists credentials as an isolation boundary
File storage	S3-compatible, per-org prefix, SSE, signed time-limited URLs	Supports deletion and residency requirements later
Background jobs	Same worker fleet, separate queues: <code>execution</code> , <code>ingestion</code> , <code>schedule</code> , <code>maintenance</code>	Ingestion must never starve execution; separate queues make that a config change
Observability	OpenTelemetry for system telemetry plus a separate first-class execution trace store	Two different consumers. Conflating them produces a customer-facing trace that leaks internals
Deployment	Containers on managed Kubernetes or equivalent; IaC; independent scaling of control plane and workers	The two scale on different axes and will diverge quickly

H.3 Deliberately not recommended yet

Per the instruction not to introduce technology without a clear reason: no microservices mesh, no Kafka (Postgres + Redis suffice until sustained event volume proves otherwise), no separate graph database (the graph is small and lives in the spec), no dedicated search cluster, no fine-tuning infrastructure, no self-hosted model serving, no multi-cloud. Introduce each only when a measured constraint demands it.

I. Data & API Model

I.1 Entities

Tenancy & identity: Organization · Workspace · User · Membership(user, workspace, role) · Role · Permission · ApiKey · SsoConfig

Agent definition: Agent · AgentDraft · AgentVersion(spec JSONB, immutable) · SpecPatch · Template · Trigger

Tools & MCP: McpServerCatalogEntry · McpConnection · Credential · ToolDefinition(snapshot + schema_hash) · ToolGrant · ConnectionHealthCheck

Knowledge: KnowledgeBase · Document · Chunk · Embedding · KnowledgeBinding · IngestionJob

Memory: MemoryStore · MemoryRecord (scope, key, value, embedding, TTL)

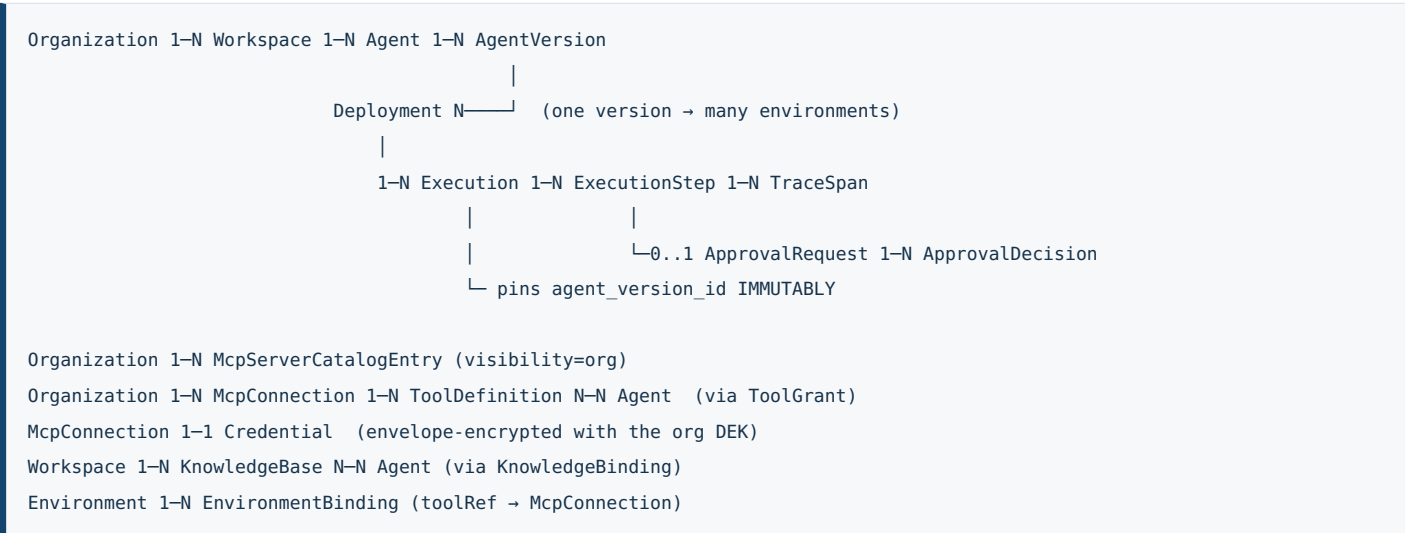
Runtime: Environment · Deployment · EnvironmentBinding(toolRef → connection) · Execution · ExecutionStep · TraceSpan · ApprovalRequest · ApprovalDecision · Conversation · Message

Evaluation: EvalSuite · EvalCase · EvalRun · EvalResult · Feedback

Governance & commercial: AuditLog · PolicyRule · UsageRecord · Notification · Alert

Marketplace (Phase 3): Listing · ListingVersion · Installation

I.2 Relationships, with the reasoning



Seven modelling decisions that are expensive to change later:

- Execution.agent_version_id is pinned and immutable.** Without it, a three-month-old trace cannot be interpreted and S16 is unsatisfiable.
- Versions are immutable; drafts are separate and singleton per agent.** Editing produces a draft; publishing produces a version; deployments reference only versions.
- Connections are bound per environment, not in the spec.** The only way promotion works without spec edits.
- ToolDefinition carries a schema_hash and a side_effect_class.** The first enables drift detection, the second drives dry-run, default approval policy and risk scoring. Both are near-impossible to backfill.
- Credentials are referenced, never embedded.** Makes specs diffable, exportable and safe to render in a UI.
- TraceSpan is partitioned by time from day one** and has its own retention policy separate from AuditLog.
- Marketplace install copies rather than references** (Phase 3). A live reference means a third party can silently change what runs in a customer's tenant.

I.3 API modules

Full endpoint listing is in the companion document [REF: §13]. The module grouping:

Module	Endpoints	Notes
auth	login, logout, refresh, me, sso initiate/callback, api-keys	
organizations / workspaces	CRUD, settings, members	
users / roles	CRUD, invite, role assignment, permission check	
agents	CRUD, duplicate, draft get/put, draft/patch , undo, validate	patch is the conversational-edit endpoint
versions	list, create (publish), get, diff	
design	generate, clarify, patch, explain, suggest-mapping, generate-tests	All streaming
mcp	catalog CRUD, connections CRUD, discover , health, oauth authorize/callback, tool grants	The E architecture, exposed
tools	catalog (filtered by grant), schema	What the Builder sees; no MCP terminology
knowledge	KB CRUD, documents upload/list/delete/reindex, query (retrieval playground), sync	
deployments	environments, deploy, list, rollback, pause/resume, triggers	
executions	run (with dryRun), invoke, list, get, steps, trace, stream , cancel, retry, bulk-retry	stream is SSE
approvals	list mine, get, approve, reject, delegate, token:signed	The token path is the out-of-app deep link
evaluation	suites, cases, run, results, compare	Phase 2
analytics	overview, per-agent, costs, errors, usage	
audit	list, filter, export	
webhooks	inbound /webhooks/:triggerToken (signature-verified, idempotent)	Public surface, hardened
admin	tenant lifecycle, fleet health, feature flags, impersonation	Vendor-side, separate auth realm

[RECOMMENDED] Conventions from day one: cursor pagination; **Idempotency-Key** required on every state-changing runtime call (trigger dedupe depends on it); RFC 7807 error format; URL-prefixed API versioning; per-tenant and per-endpoint rate limits; SSE not polling for live state.

J. Codebase Gap Analysis

J.1 Status: BLOCKED

No codebase was provided. The workspace contains only `PRODUCT_VISION_AGENTS.pdf` . I searched for repositories, source trees, archives, manifests, schemas and migrations and found none. Producing a gap analysis from nothing would be fabrication, so instead this section gives you the inspection protocol so the analysis can be completed the moment access exists.

J.2 What to provide

Any one of: read access to the repository, a git bundle or archive of the source tree, or a `git ls-files` listing plus the key files below.

Priority files, in order of diagnostic value: 1. `package.json` / `pyproject.toml` / `go.mod` (stack, dependencies, whether an agent framework is already in use) 2. Database migrations or schema (`prisma/schema.prisma` , `alembic/` , `*.sql`), **the single most informative artifact**, because the data model reveals which of the seven decisions in I.2 were already made 3. Directory tree to depth 3 4. API route definitions or OpenAPI spec 5. Anything under `agent/` , `runtime/` , `executor/` , `workflow/` , `mcp/` , `tools/` , `auth/` 6. `docker-compose.yml` / IaC (infrastructure assumptions) 7. Test directory structure (what quality bar exists today)

J.3 Classification rubric I will apply

Class	Definition
Existing	Implemented, tested, matches the blueprint's intent, safe to build on
Partially implemented	Present but incomplete, untested, or diverging from the target design; will state what remains and whether to extend or replace
Missing	Not present
Unclear	Present but I cannot determine intent or correctness without the author; will list the specific question

Applied across all ten modules (M1 to M10), plus the five foundational decisions (D1 to D5).

J.4 The specific architectural risks I will test for

These are derived from this blueprint, ranked by cost-to-fix if already wrong. This list is the actual value of the exercise, and it is usable by your team today even before I see the code.

#	Risk to check	Why it is expensive	Cheap diagnostic
R1	Two representations of an agent (a canvas model and a separate runtime model)	Violates D1. Reconciling later is a rewrite of both	Is there exactly one schema that both the UI and the executor consume?
R2	Synchronous or in-memory execution with no checkpointing	Violates D2. Cannot support S15 approvals. Rewrite of every executor	Search for a <code>waiting</code> / <code>suspended</code> execution state and any timer or signal mechanism. If absent, D2 has not been addressed
R3	Tool authorization decided inside prompts or by the model	Violates D3. Model is the security boundary	Is there a single choke point every tool call passes through, or are tools called directly from executors?
R4	Tenant filtering via application-level <code>WHERE org_id = ?</code>	Violates D4. One forgotten clause is a breach	Does the schema have RLS policies? Is <code>org_id</code> even on every tenant table?

#	Risk to check	Why it is expensive	Cheap diagnostic
R5	Traces written as unstructured logs	Violates D5. S16 becomes unanswerable	Is there a <code>trace_span</code> or <code>execution_step</code> table, or only log statements?
R6	Credentials stored plaintext or with a single global key	Blocks enterprise sale entirely	Grep for encryption at the credential layer; check for per-org key derivation
R7	Connection IDs embedded in the agent definition	Makes staging-to-production promotion impossible	Does the spec reference tools abstractly or point at a specific connection?
R8	Tool integrations hard-coded per provider rather than behind one interface	Guarantees the integration long tail becomes headcount	How many places construct an HTTP call to a specific vendor?
R9	Mutable agent records with no version table	Breaks rollback, audit and trace interpretability	Is there an <code>agent_versions</code> table with an immutability constraint?
R10	No side-effect classification on tools	Blocks dry-run, risk scoring and default approval policy	Does any tool metadata distinguish read from write?
R11	Vector data not namespaced per tenant	Cross-tenant knowledge leakage, the worst failure mode	Check the retrieval filter: is the tenant filter applied at the store or after?
R12	Frontend calling model providers directly	Leaks keys, bypasses policy, metering and tracing	Any provider SDK in the frontend bundle

I will also identify **reusable code**: auth and tenancy scaffolding, UI component library, connector implementations worth wrapping as `in_process` MCP servers, ingestion or parsing utilities, and test infrastructure. In my experience with projects at this stage, auth scaffolding and UI components are almost always reusable, while any existing execution engine is the most likely thing to need replacement.

J.5 What I will deliver once unblocked

A module-by-module classification table, a decision-by-decision (D1 to D5) verdict, a keep / extend / replace recommendation per component with rationale, a migration path for anything classified replace, and a revised roadmap in **L** adjusted for what already exists.

K. MVP / Phase 2 / Phase 3

K.1 A contradiction in the vision that must be resolved

[GAP] S25 says "we should NOT attempt to prototype every possible feature" and defines a single golden path. S26 then lists a first release spanning creation, configuration (models, tools, MCP, knowledge, memory, output, guardrails, approval), test datasets, staging and production with rollback, full monitoring, RBAC, isolation and audit.

[ASSUMPTION] S25 describes a **prototype** and S26 describes a **commercial GA**. They are different milestones with several quarters between them. This blueprint treats them as such. If that assumption is wrong, scope must be renegotiated before Workstream 1 starts.

K.2 MVP

Thesis: the MVP proves the riskiest assumption, it does not demonstrate the most features. The riskiest assumption is that a business user can describe a process and end up with a deployed, trusted, working agent (S30). Everything not serving that proof is deferred.

Constraint: one department, one design partner. The vision names Finance, HR, Sales, Support, IT and Legal. Attempting all six at MVP guarantees shallow quality everywhere. HR onboarding is already the vision's own chosen scenario (S25), which makes it the natural choice.

Success metric: a business user at the design partner creates and deploys an agent that runs successfully against their real systems, in one working session, without engineering help. This is S30 made measurable.

In MVP	Why it cannot be deferred
Agent Spec v1 schema + validation	D1. Everything depends on it
Durable execution, 7 node types (trigger, agent, knowledge, condition, tool, approval, notification)	D2 + exactly the S25 golden path
NL generation → validated spec, streaming, with repair loop	The core hypothesis
Canvas: render, edit config, insert/delete/reconnect, TODO badges	"See and edit" is half the promise
Conversational patching with visible diff	The interaction model that differentiates this
3 to 5 HR templates	Carries generation quality; shortens time-to-value
MCP layer: catalog, connections, 4 transports, OAuth + API key, discovery, per-agent grants	The integration strategy; core, not a feature
Generic HTTP connector + 2 to 3 partner-specific connectors	Proves integration without unbounded build
Knowledge: upload, ingest, retrieve, cite	Pillar 4, required by the golden path
Human approval: approver/role, timeout, escalation, conditions, rejection, audit, out-of-app link	S15 in full; the enterprise differentiator
Dry-run / mocked execution	[RECOMMENDED, not in the vision] Without it, testing "Create Employee Record" creates a real employee. Activation dies
Live canvas execution (SSE)	S13 calls this important to the experience
Full execution trace answering the S16 questions	Trust; retrofitting is prohibitively expensive
Versioning, staging + production, deploy, rollback, pre-deploy validation gate	S26; makes deploying feel safe
Cost, token and latency capture	S9 shows it; metering must be right from the start
RBAC: 4 system roles	Minimum viable governance
Tenant isolation (RLS, namespaces, per-org DEKs) + CI isolation suite	D4; architectural, not a feature
Immutable audit log	D5 sibling; same reasoning

In MVP	Why it cannot be deferred
Basic monitoring dashboard	Operating a deployed agent requires it

Explicitly out of MVP: marketplace (no value with one partner, unsolved portability), multi-agent (the vision itself says "eventually"), memory beyond scratchpad and conversation (undefined), evaluation framework (manual testing suffices to prove the thesis), SSO/SCIM and custom roles (needed to sell, not to validate), broad connector library (escape hatches cover it), advanced guardrails beyond PII redaction, public API and SDK, billing, WhatsApp and mobile.

K.3 Phase 2: make it sellable

Theme: from "works for one partner" to "passes enterprise procurement and security review."

Evaluation framework with test datasets, scoring and deploy gating · memory (once defined) · guardrails configuration · SSO (SAML/OIDC) and SCIM · custom roles and granular grants · webhook and connector-event triggers · 10 to 15 connectors driven by real pipeline · alerting, error queue, bulk retry, step-level debugging · cost budgets and spend controls · AI failure triage and improvement recommendations · audit export and SIEM streaming · usage metering to billing · retention controls and GDPR tooling · knowledge auto-sync · permission mirroring · SOC 2 readiness.

Every item is either a procurement blocker or an operational necessity beyond a handful of agents. None is needed to prove the concept.

K.4 Phase 3: make it a platform

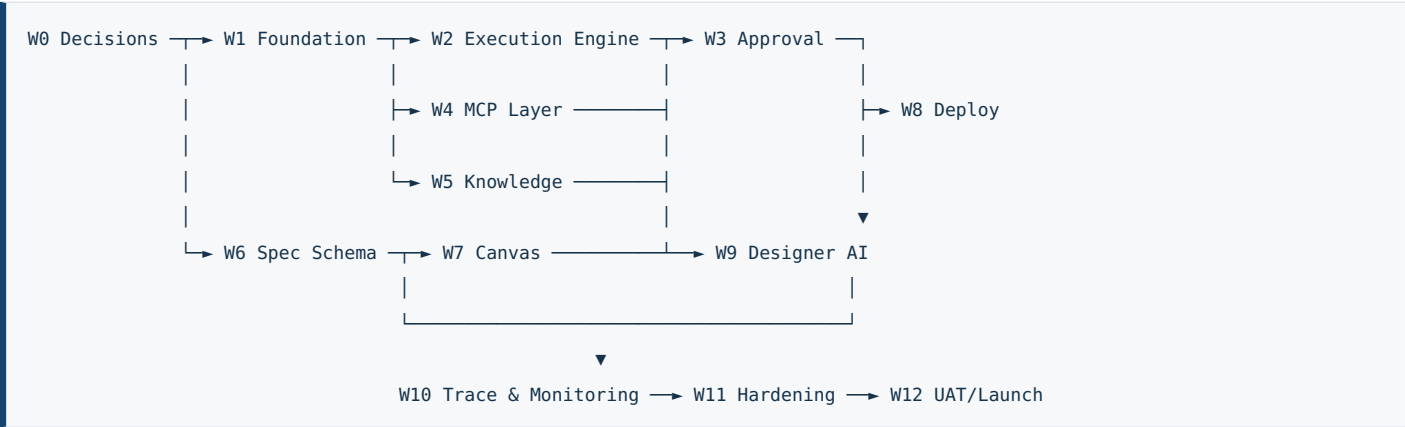
Marketplace with configuration contracts and third-party publishing · multi-agent orchestration (S6, S27) · platform-as-MCP-server plus public API and SDK · policy-as-code governance · data residency and VPC/self-hosted runtime · model routing with evaluation-backed cost optimization · agent-to-agent collaboration and the agent workforce view · business-outcome analytics · behavioural anomaly detection.

Each depends on a mature core. The marketplace in particular needs many working agents, a proven portability model and a certification process before it creates value rather than support burden.

L. Implementation Roadmap (dependency order)

Sequenced so that **risk is retired early** and there is always a demoable path. Durations assume roughly 6 to 8 people (1 PM, 1 designer, 3 to 4 engineers including one with AI depth, 1 QA, part-time DevOps and security). **[ASSUMPTION]**, adjust to actual staffing.

L.0 Dependency graph



Two things to note about this graph. First, **W6 (Spec Schema) is on the critical path for almost everything** and must be finished before W7 or W9 begin. Second, **W9 (Designer AI) is deliberately late**, which feels counterintuitive because it is the headline feature. It is late because it produces specs that nothing can execute until W2, W4 and W5 exist. Building the demo first is the most common failure pattern in platforms of this shape.

L.1 Workstreams

WS	Name	Depends on	Duration	Exit criteria
W0	Decisions & spikes	(none)	2 to 3 wks	All 15 P0 questions answered. Three spikes complete: (a) NL-to-spec quality measured on 20 prompts, (b) durable execution suspend/resume/timeout proven, (c) tenant isolation proven end to end. Go/no-go on the generation approach. Build-vs-buy decided for the engine
W1	Foundation	W0	3 wks	Org/workspace/user/role model with RLS live. Auth working. Audit log writing. CI with the cross-tenant isolation suite green. Deployment pipeline exists
W6	Agent Spec schema	W0	1 to 2 wks (parallel with W1)	v1 JSON Schema frozen, validator implemented, migration mechanism defined, 3 hand-written example specs pass validation

WS	Name	Depends on	Duration	Exit criteria
W2	Execution engine	W1, W6	4 wks	A hand-written spec executes end to end with 3 node types (trigger, tool, notification). Checkpointing, retry, idempotency, budgets working. Worker can be killed mid-run and resume. This is the highest-risk workstream
W4	MCP layer	W1, W2	4 wks (overlaps W2 from wk 2)	Catalog, connection, credential broker with per-org DEKs, all 4 transports, dynamic discovery with drift detection, per-agent grants, policy decision point in the call path. One real third-party MCP server and one in_process first-party connector both working through the identical path
W3	Approval workflow	W2	2 wks	Suspend, notify, signed single-use link, decide, resume, timeout, escalation, immutable decision record. All S15 elements
W5	Knowledge & RAG	W1	3 wks (parallel)	Upload → parse → chunk → embed → index → retrieve → cite, with per-tenant namespaces and ingestion status visible
W7	Canvas	W6	4 wks (starts after W6)	Render a spec, edit node config, insert/delete/reconnect, validation errors inline, TODO badges, diff review UI
W9	Designer AI	W6, W4, W5, W7	3 wks	Generate, clarify, patch-not-regenerate, explain, repair loop, streaming. Measured against the W0 baseline
W10	Trace, live view & monitoring	W2	3 wks (overlaps)	Structured spans, trace UI answering all ten S16 questions, SSE live canvas, cost rollup, dashboard
W8	Deploy, versioning, environments	W2, W4	2 wks	Publish version, environment bindings, deploy, validation gate, rollback, in-flight version pinning
W11	Hardening	all	3 wks	Security red team, chaos and durability testing, load test, performance, accessibility, error handling pass

WS	Name	Depends on	Duration	Exit criteria
W12	UAT & launch	W11	4 wks	Design partner completes the full journey unaided; runbooks, SLOs, on-call, backup restore verified, security sign-off

Indicative total: 22 to 26 weeks to a controlled launch, assuming W0 answers arrive on time. **[ASSUMPTION]** This is an estimate, not a commitment, and it will change materially once the existing codebase is assessed (J).

L.2 Continuous tracks (not sequential)

- **AI quality measurement** from W0 onward: nightly benchmark, baselines, regression alerts. Not a phase, an instrument.
- **Security**: threat model in W1, SOC 2 gap analysis starting mid-build, not after launch.
- **Design partner engagement**: continuous, not just at W12.
- **QA**: golden path and isolation suites automated from the first sprint, not at the end.

L.3 Sequencing rules I would enforce

1. **Nothing user-facing before W2 works.** A vertical execution slice by end of W2 is the project's first real milestone.
2. **W6 before W7 and W9.** Both consume the spec; changing it after they exist means rewriting both.
3. **W4's policy decision point ships with W4, not after.** Adding authorization to an existing invocation path means auditing every call site.
4. **W10's trace capture ships alongside W2,** even if the trace UI comes later. Instrumentation added retroactively always has gaps.
5. **Do not build all the UI before any of the runtime.** This is the single most common way projects of this shape fail.

M. Risks

Ranked by expected impact. Mitigations are concrete, not platitudes.

M.1 Security risks

Risk	Severity	Mitigation
Cross-tenant data leakage (agents, knowledge, credentials, traces)	Company-ending	D4: RLS, per-tenant vector namespaces, per-org DEKs. Automated adversarial isolation suite in CI on every commit. Never rely on review
Prompt injection → unauthorized real-world action	Critical	D3: policy engine outside the model. Untrusted-content isolation for retrieved docs, tool outputs and MCP tool descriptions. Approval gates on irreversible tools. Execution budgets. Dedicated red-team suite
Credential compromise across tenants	Critical	Per-org DEKs under a KMS root, just-in-time decryption at the transport layer, never in traces/logs/model context, automated secret scanning of stored spans
Malicious or compromised MCP server	High	Admin approval before use, schema pinning with drift alerts, sanitized tool descriptions, egress allowlist, sandboxed stdio, response caps
Privilege escalation via agents	High	Effective-permission ceiling bounded by the invoking user, action-level grants, segregation of duties
Forged or replayed approvals	High	Signed, single-use, expiring tokens; already-decided detection; immutable decision records with payload hashes

M.2 Technical risks

Risk	Mitigation
NL-to-spec quality insufficient to be trustworthy (the core promise)	Curated pattern library carries quality, not raw model capability. Constrained generation + repair loop. Human review always required. Measure from W0 and continuously
Durable execution built wrong → the rewrite S21 forbids	Spike it in W0, decide build-vs-buy on evidence, prove kill-and-resume before building executors on top
Two agent representations diverge (D1 violated)	One schema, enforced by module dependency rules and a CI check that the canvas and runtime import the same types
Integration long tail becomes headcount	Three-tier strategy: MCP-first, few curated connectors, generic HTTP/SQL/webhook escape hatches
Systems behind customer firewalls unreachable	[GAP] The vision assumes cloud-to-cloud. Either scope a customer-installed gateway (E.3 stdio /internal MCP gateway is the natural vehicle) or explicitly restrict the initial target market to cloud-native systems
Trace volume and cost explosion	Time-partitioned storage, span sampling for non-critical events, tiered retention, cost modelled before launch
Model provider change breaks behavior silently	Pinned model versions, nightly regression benchmark, provider abstraction with fallback, model upgrades treated as releases

M.3 Product and UX risks

Risk	Mitigation
The magic-then-cliff problem: generation delights, then twelve configuration blockers appear	TODO badges with one-click resolution, mockable tools so a test run is possible before any real integration exists
Canvas complexity creep re-creates the "25 settings" experience S11 rejects	Progressive disclosure by persona; the word "MCP" never appears in the Builder's UI
Users cannot predict agent behavior	Deterministic conditions and structure visually distinct from model-driven agent nodes, so users know exactly where variability lives
Approval fatigue → rubber-stamping	Show consequences not payloads; surface "this approval is always approved" as a removal candidate
Positioning collapse: a platform for every department appeals to no one	One department, one design partner for MVP. Expand only after repeatable wins

M.4 Scalability risks

Risk	Mitigation
Noisy neighbour	Per-org concurrency caps, fair queue scheduling, isolated worker pools for large tenants
Execution and trace tables outgrowing Postgres	Time partitioning from day one, archival tiering, separate trace store if volume demands
Vector index growth and re-index cost	Per-org namespaces, incremental indexing, embedding-migration path planned before the first model change
Canvas performance on large graphs	Virtualized rendering, soft node-count guidance
Long-running executions holding resources	Durable suspension holds no compute; this is a direct benefit of D2

M.5 Business risks

Risk	Mitigation
Heavily contested market (Copilot Studio, Agentforce, ServiceNow, n8n, LangGraph Platform, vertical AI startups)	Differentiate on the operating layer, which incumbents bolt on late, and on being system-of-record agnostic, which suite vendors structurally cannot be
Long enterprise sales cycles exhaust runway	Design a low-risk entry point (read-only agents, no write actions, minimal security surface) that can land before full procurement
Buyer unidentified → product built for nobody	N.1 questions 1 and 2 answered before UX design starts

N. Open Questions

N.1 P0: block architecture or MVP scope

#	Question	Blocks
1	Primary user: business analyst, AI CoE team, or developer? (S3 vs S26 contradiction)	All UX, W7, W9
2	Buyer and pricing metric (per agent / execution / seat / consumption)?	Metering design in W1
3	Target customer size and industry?	Security posture, deployment model
4	Which single department first?	MVP scope, templates, connectors
5	Is there a committed design partner with real systems?	W12, and MVP validity
6	Build or buy the durable execution engine?	W2, the largest single estimate
7	What exactly is "Memory"? (in the S4 core diagram and S26 release, never defined)	Data model, D.5
8	What does "Evaluate" mean, and does a failing eval block deployment?	W8 gate, Phase 2 scope
9	Full trigger taxonomy for v1?	W2, W8
10	How does an end user actually interact with a deployed agent? Portal, chat, embedded, API-only?	Entire surface is unscoped; for HR support and IT helpdesk this is the product
11	Whose credentials does an agent use at runtime: service account, delegated, or invoking user?	W4 connection model, OAuth flows, audit story
12	What is a "Workspace": department, team, project, or environment?	W1 data model
13	Cloud-only, or VPC / on-premise?	Fundamental architectural fork
14	How do we reach systems behind a customer firewall?	Likely requires a gateway component, currently unscoped
15	Which model providers; can customers bring keys or require a region?	AI layer, commercial guarantees

N.2 P1: block Phase 2 planning

Data residency requirements · required compliance certifications and dates · trace retention for business data · availability and latency SLOs · target scale for capacity planning · marketplace first-party vs third-party · code escape hatches (custom function nodes) · how much of a generated agent's prompts a user may edit.

N.3 Contradictions to resolve

#	Contradiction	Where
C1	User "should not need to understand MCP" vs "Configure MCP" in the first release	S3 vs S26. Proposed resolution: persona split (Technical Steward sees MCP, Builder never does)
C2	"Do not prototype every feature" vs a first release spanning six major areas	S25 vs S26. Proposed resolution: two milestones (K.1)

#	Contradiction	Where
C3	Record "what data was sent" vs GDPR data minimisation for PII	S16 vs privacy law. Proposed resolution: redacted trace by default, full payload separately access-controlled with shorter retention
C4	Strict tenant isolation vs a marketplace sharing artifacts across tenants	S20 vs S18. Proposed resolution: copy-on-install, never live reference
C5	"Deterministic workflows" vs autonomous agent reasoning	S23 vs S6. Proposed resolution: deterministic conditions and structure; model variability confined to agent nodes (D.1 rule 3)

N.4 Missing requirements (in the vision, not in this blueprint)

End-consumer surface · SSO and SCIM · data residency · compliance certifications · firewall reachability · pricing and metering · vendor-side admin tooling · dry-run/simulation · marketplace publishing and liability model · SLOs · disaster recovery and RPO/RTO.

O. Recommended Next Steps

O.1 Immediate (this week)

1. **Send me the codebase** (J.2) so section **J** can be completed and **L** re-baselined against reality. Until then all durations in **L** are estimates against a greenfield assumption.
2. **Convene a 2-hour decision session on the 15 P0 questions.** Most are business decisions, not research. Answering 11 of them requires only a decision-maker in a room.
3. **Confirm or deny the K.1 assumption** that S25 and S26 are two different milestones. If they are meant to be one, scope must be renegotiated now.
4. **Start W0 spike (b): durable execution.** It is independent of every P0 answer and it is the largest technical unknown.

O.2 Weeks 1 to 3 (W0)

Run the three spikes in parallel because each can invalidate the plan: NL-to-spec quality on 20 real prompts (tests the core hypothesis), durable execution with approval suspend/resume/timeout (tests D2), tenant isolation end to end (tests D4). Produce an ADR set committing D1 to D5. Secure the design partner.

O.3 Then follow L in dependency order

W1 + W6 in parallel → W2 → W4 and W5 in parallel → W3, W7 → W9, W10 → W8 → W11 → W12.

O.4 What can start today regardless of the open questions

Roughly 40% of the work is invariant to every P0 answer, so there is no reason to be blocked:

- The Agent Spec v1 schema (W6): the structure does not depend on who the user is
- The durable execution engine (W2): invariant to product decisions
- Tenancy, RLS, auth scaffolding, audit log (W1)
- The MCP layer (W4): the architecture in **E** is complete and stable
- Knowledge ingestion and retrieval (W5)
- CI, the cross-tenant isolation suite, and the AI quality benchmark harness

O.5 What must NOT start until the questions are answered

- Any UX design (blocked on Q1: who is the user)
 - Memory implementation (blocked on Q7: it is undefined)
 - The evaluation framework (blocked on Q8)
 - The end-consumer surface (blocked on Q10: it does not exist as a requirement yet)
 - Connector selection (blocked on Q4 and Q5: driven by the design partner's actual stack)
 - Deployment topology and any gateway component (blocked on Q13 and Q14)
-

Is the project sufficiently understood to start implementation?

Partially. Foundation work can start now; product-surface work cannot.

Being precise, because a blanket yes or no would be misleading:

Yes for the foundation. The five architectural decisions (D1 to D5) are well-grounded in the vision and do not depend on any unanswered question. The MCP architecture in **E** is complete and buildable as specified. The data model in **I** is stable. Workstreams W1, W2, W4, W5 and W6 can begin immediately, and they represent roughly 40% of the MVP effort and essentially all of the technical risk. Starting here is also the correct sequencing on its own merits, because it retires risk earliest.

No for the product surface. Fifteen P0 questions are open, and three of them are severe enough that building against a guess would produce work that has to be thrown away: - **Q1 (who is the primary user)** determines the entire UX. The vision contradicts itself on this (S3 vs S26). - **Q10 (how an end user interacts with a deployed agent)** is not merely unanswered, it is entirely absent from the vision. For the HR support and IT helpdesk use cases the vision itself names, this is the whole product. - **Q7 (what "Memory" means)** is listed as one of five core platform primitives and as a first-release item, with no definition anywhere.

And genuinely unknown until I see the code. Section **J** is blocked. The gap analysis could materially change the roadmap in either direction: existing auth, tenancy and UI scaffolding could remove weeks, while an existing synchronous execution engine or a dual agent representation (risks R1 and R2) could mean replacing the component the whole platform rests on. I would not commit to the L timeline before that assessment.

My recommendation: answer the P0 questions in a single focused session this week, send me the codebase, and start W0's three spikes in parallel immediately. The spikes are the correct first move whatever the answers turn out to be, and by the time they finish you will have both the decisions and the gap analysis needed to commit to the full plan with confidence.

I am stopping here and will not touch any code until you approve.